

Procesamiento de Lenguajes (PL)

Curso 2025/2026

Practica 2: analizador sintactico SLR

Fecha y metodo de entrega

La práctica debe realizarse de forma individual, y debe entregarse a través del servidor de prácticas del DLSI antes de las **23:59 del 20 de marzo de 2026**. Los ficheros fuente en Java se comprimirán en un fichero llamado "plp2.tgz", y no debe haber directorios (ni package) en él, solamente los fuentes en Java.

Al servidor de prácticas del DLSI se puede acceder desde la web del DLSI (<http://www.dlsi.ua.es>), en el apartado "Entrega de prácticas". Una vez en el servidor, se debe elegir la asignatura PL y seguir las instrucciones.

Descripcion de la practica

La práctica consiste en diseñar un analizador sintáctico ascendente, utilizando el algoritmo de análisis por desplazamiento-reducción, y construyendo las tablas con el método SLR. La gramática sería la siguiente:

(1)	<i>S</i>	→	class <i>id</i> lbra <i>M</i> rbra
(2)	<i>M</i>	→	<i>Fun</i> <i>M</i>
(3)	<i>M</i>	→	<i>S</i> <i>M</i>
(4)	<i>M</i>	→	ε
(5)	<i>Fun</i>	→	fun <i>id</i> lbra <i>M</i> <i>Cod</i> rbra
(6)	<i>DV</i>	→	<i>Tipo</i> id
(7)	<i>Tipo</i>	→	int
(8)	<i>Tipo</i>	→	float
(9)	<i>Cod</i>	→	<i>Cod</i> pyc <i>I</i>
(10)	<i>Cod</i>	→	<i>I</i>
(11)	<i>I</i>	→	<i>DV</i>
(12)	<i>I</i>	→	lbra <i>Cod</i> rbra
(13)	<i>I</i>	→	id asig <i>E</i>
(14)	<i>I</i>	→	print <i>E</i>
(15)	<i>E</i>	→	<i>E</i> opas <i>F</i>
(16)	<i>E</i>	→	<i>F</i>
(17)	<i>F</i>	→	numentero
(18)	<i>F</i>	→	numreal
(19)	<i>F</i>	→	id

Ten en cuenta las siguientes cuestiones:

1. Construye las tablas de análisis utilizando el método SLR, explicado en clase de teoría (deben salir 39 estados, del 0 al 38). Después, implementa el analizador a partir de esas tablas. Esta gramática no debes modificarla, la recursividad por la izquierda (y los factores comunes por la izquierda) no impiden construir un analizador ascendente.
2. Como en el analizador de la práctica 1, en caso de que se encuentre un error, se debe emitir el siguiente mensaje:

```
Error sintactico (f,c): encontrado 'token', esperaba ...
```

donde f es la fila, c es la columna, y token es el carácter incorrecto, y después de esperaba debe aparecer una lista con los tokens que se esperaba, mostrando para cada token la cadena que aparece en la tercera columna de la tabla de los componentes léxicos de la práctica 1. Si aparece algún error se debe terminar la ejecución con `System.exit(-1)`

Ejemplo:

Error sintactico (4,5): encontrado 'hola', esperaba ; + - }

Los tokens esperados deben aparecer en el mismo orden que en la tabla de la práctica 1, para lo que la tabla ACCIÓN del analizador debe tener los tokens en este mismo orden:

class, id, lbra, rbra, fun, int, float, pyc, asig, print, opas, numentero, numreal.¹

Si el token encontrado es el final del fichero, el mensaje de error debe ser:

Error sintactico: encontrado fin de fichero, esperaba ...

3. La práctica debe tener varias clases en Java:

- La clase plp2, que tendrá solamente el siguiente programa principal (y los import necesarios):

```
class plp2 {
    public static void main(String[] args) {

        if (args.length == 1)
        {
            try {
                RandomAccessFile entrada = new RandomAccessFile(args[0], "r");
                AnalizadorLexico al = new AnalizadorLexico(entrada);
                AnalizadorSintacticoSLR aslr = new AnalizadorSintacticoSLR(al);

                aslr.analizar();
            }
            catch (FileNotFoundException e) {
                System.out.println("Error, fichero no encontrado: " + args[0]);
            }
        }
        else System.out.println("Error, uso: java plp2 <nomfichero>");
    }
}
```

- La clase AnalizadorSintacticoSLR, que tendrá al menos el método analizar para analizar la cadena de entrada.
- Las clases Token y AnalizadorLexico de la práctica 1, convenientemente adaptadas ambas al lenguaje de esta práctica. Ten en cuenta que desaparecen los tokens pari, pard, dosp, oprel, opmul, if, else y fi, pero el analizador léxico debe seguir ignorando los comentarios.

4. La salida de la práctica debe ser los números de las reglas que ha ido aplicando el analizador sintáctico. Por ejemplo, ante este fichero de entrada:

```
class Main {
    fun main {

        class A {
        }

        /* codigo main */

        print 1+2-3.4
    }
}
```

la salida de la práctica debería ser:

1 2 4 5 10 14 15 18 15 17 16 17 3 4 1 4

Nota: como el analizador SLR obtiene la inversa de una derivación por la derecha de la cadena de entrada, es necesario almacenar las reglas por las que se reduce en un vector y luego imprimir el vector en orden inverso para que salga una derivación correcta².

¹Es aconsejable en la clase Token que ordenes los tokens siguiendo este orden.

²También se puede utilizar una pila para almacenar los números de regla, que facilita que los números salgan en orden inverso.